Agents-Refund

How I set up my Python project with uv

A simple step-by-step guide

> This guide shows **what you already did** to set up the project, **what each command means**, and **how to add more packages** later. The steps come from your own PowerShell history on 5 November 2026.

1. What is uv?

uv is a tool for Python projects. It does three jobs for you:

• It **creates a virtual environment** (a private box for your project's packages).
• It **installs packages**, like pip does, but much faster.
• It **writes down** which packages and versions your project uses, so it works the same on every computer.

> **Is uv a virtual environment?** No. uv is the **tool**. The virtual environment is the **.venv** folder that uv made inside your project. Think of uv as the builder, and .venv as the room it built.

2. What is a virtual environment?

A virtual environment is a folder that holds the packages for **one project only**.

Without it, every project on your computer shares the same packages. If project A needs version 1 of a package and project B needs version 2, they break each other. With a virtual environment, each project has its own copy, so they never fight.

In your project, the virtual environment is the `.venv` folder. It uses Python 3.12.10. You never edit it by hand, and you never upload it to GitHub.

3. What you did, step by step

These are the commands you typed, in order. Each one has a short explanation.

Step 1. Check that Python and pip work

```
python --version
pip --version
```

This shows which Python you have. You have **Python 3.12**. pip is Python's normal package installer.

Step 2. Install uv

```
pip install uv
uv --version
```

This installed the uv tool using pip. The second command checks it worked. You now have **uv 0.13.7**. You only do this once per computer, not once per project.

Step 3. Open your project folder

```
cd C:\Users\User\OneDrive\Documents\Agents-Refund
```

All the next commands must run **inside** the project folder.

Step 4. Start a new uv project

```
uv init
```

This made the starting files: `pyproject.toml`, `README.md`, `.python-version`, `.gitignore` and the `src` folder with a small hello-world program.

Step 5. Create the virtual environment

```
uv venv --python 3.12
```

This made the `.venv` folder, using Python 3.12.

Step 6. Turn the virtual environment on (activate it)

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.venv\Scripts\Activate.ps1
```

The first line lets PowerShell run the activate script, **only for this window**. The second line turns the environment on. You will see **(Agents-Refund)** at the start of your prompt. *Tip:* with uv you often do not need to activate at all. `uv run` uses .venv for you (see section 5).

Step 7. Add the packages your project needs

```
uv add dotenv langchain langchain-openai
```

This is the most important command. In one go, it:
• adds the package names to `pyproject.toml`
• works out exact versions and saves them in `uv.lock` (48 packages, because these packages

need other packages)
• installs them all into `.venv`

Step 8. Sync and lock

```
uv sync
uv lock
```

`uv sync` makes .venv match the lock file. `uv lock` rewrites the lock file. **You did not need these this time**, because `uv add` already did both. They do no harm.

Step 9. Protect your secrets

```
# you made .env and .env.example, then added this line to .gitignore:
.env
```

`.env` holds your real API key. Adding it to `.gitignore` means Git will never upload it. Good job, this is done.

4. Your project files now

File or folder	What it is	Upload to GitHub?
`pyproject.toml`	The list of packages you asked for, and project info.	Yes
`uv.lock`	The exact version of every package. Made by uv. Do not edit.	Yes
`.python-version`	Says this project uses Python 3.12.	Yes
`.venv/`	The virtual environment with installed packages.	No (already in .gitignore)
`.env`	Your real secrets, like API keys.	**Never** (already in .gitignore)
`.env.example`	Same setting names as .env, but with empty values.	Yes, once the value is empty
`.gitignore`	The list of files Git must ignore.	Yes
`src/agents_refund/`	Your Python code. Right now a hello-world program.	Yes
`agents/models.py`	An empty file, ready for your agent code.	Yes

One thing to fix: your `.env.example` still has a long value after `GEMINI_API_KEY=`. That looks like a real key. This file **will** be uploaded, so make it empty:
`GEMINI_API_KEY=`
If it is a real key, also make a new key at aistudio.google.com/apikey and delete the old one, because the folder is synced to OneDrive.

5. How to run your code

```
uv run agents-refund
```

This runs your program inside .venv. Right now it prints *Hello from agents-refund!*

To run any Python file:

```
uv run python agents\models.py
```

`uv run` always uses the right Python and packages, even if you did not activate .venv.

6. How to add more packages later

Add a package

```
uv add langchain-google-genai
```

This updates pyproject.toml, updates uv.lock, and installs it. Always use this, **not** pip install.

Add a package only for development (for example tests)

```
uv add --dev pytest
```

Remove a package

```
uv remove langchain-openai
```

See what is installed

```
uv tree
```

Update packages to newer versions

```
uv lock --upgrade
uv sync
```

> **Why not pip install?** `pip install` puts the package in .venv but does **not** write it in pyproject.toml or uv.lock. Then the project breaks on another computer. `uv add` does all three.

7. Setting up on a new computer

1. Install Python 3.12 and uv:

```
pip install uv
```

2. Download your project from GitHub and go into it:

```
git clone <your-repo-url>
cd Agents-Refund
```

3. Install the exact same packages:

```
uv sync
```

4. Make your private .env from the template, then type your real key into it:

```
copy .env.example .env
```

8. Cheat sheet

Command	What it does	When to use it
`uv init`	Starts a new project.	Once, at the start.
`uv venv`	Makes the .venv folder.	Once (uv add and uv sync also make it if missing).
`uv add NAME`	Adds and installs a package.	Every time you need a new package.
`uv remove NAME`	Removes a package.	When you stop using one.
`uv sync`	Makes .venv match uv.lock.	After git clone or git pull, or if .venv breaks.
`uv lock`	Saves exact versions to uv.lock.	Rarely. After editing pyproject.toml by hand.
`uv run ...`	Runs a command inside .venv.	Every time you run your code.
`uv tree`	Shows installed packages.	To check what you have.

Remember: uv is the tool. .venv is the box. pyproject.toml is your wish list. uv.lock is the exact shopping receipt.